Matplotlib: krijimi i grafikëve 2D

Alion Emini

Kurs i Programimit në Python

kurse.informatika

October 1, 2025

Contents

1 Matplotlib: krijimi i grafikëve 2D **5**

1.1 Çfarë është `matplotlib`? Instalimi . . . 5

1.2 Importimi i modulit `pyplot` . . . 6

1.2.1 Forma e drejtpërdrejtë . . . 6

1.2.2 Me alias (praktika më e përdorur) . . . 6

1.3 Grafik vijor (Line Graph) me `plot` . . . 6

1.3.1 Grafiku 1: Shembull bazë . . . 6

1.3.2 Grafike 2: Shtimi i titullit, etiketave dhe rrjetës . . . 7

1.3.3 Grafiku 3: Përshtatja e kufijve të boshtit X dhe Y . . . 8

1.3.4 Grafiku 4: Përshtatja e tick-ëve me `xticks` / `yticks` . . . 9

1.3.5 Grafiku 5: Vendosja e markuesve (markers) në pikat e të dhënave . . . 10

1.3.6 Grafiku 5 dhe 6: Shembuj shtesë me `annotate`, `color`, `linestyle` . . . 11

1.4 Grafik me kolona (Bar Chart) . . . 13

1.4.1 Gjerësia e kolonave (*bar width*) . . . 14

1.4.2 Ndryshimi i ngjyrave të kolonave . . . 14

1.4.3 Titulli, etiketat dhe *tick*-ët e personalizuar . . . 14

1.5 Grafik rrethor (Pie Chart) . . . 16

1.5.1 Etiketa të sektorëve dhe titulli i grafikut . . . 16

1.5.2 Ndryshimi i ngjyrave të sektorëve . . . 17

1.6 Ndarja e figurës në nënfigura (Subplots) . . . 17

Chapter 1

Matplotlib: krijimi i grafikëve 2D

Objektivat e kapitullit

- **Matplotlib bazë**: importimi `import matplotlib.pyplot as plt`.
- **Grafik vijor**: ndërtimi me `plot(x, y)` dhe shfaqja me `show()`.
- **Dekorimi i grafikëve**: `title()`, `xlabel()`, `ylabel()`, `grid()`.
- **Boshtet dhe tick-ët**: përcaktimi i kufijve me `xlim()/ylim()` dhe personalizimi i tick-ëve me `xticks()/yticks()`.
- **Stili i vijës dhe pikave**: përdorimi i `marker`, `color`, `linestyle`, `linewidth`, `alpha`.
- **Grafik me kolona (Bar Chart)**: `bar()` me `width` dhe `color`, si dhe etiketime të personalizuara të boshtit.
- **Grafik rrethor (Pie Chart)**: `pie()` me `labels`, `colors` dhe `title()`.
- **Nënfigura (Subplots)**: ndarja e figurës me `subplot()` dhe ruajtja me `savefig()`.

1.1 Çfarë është `matplotlib`? Instalimi

Përkufizim

matplotlib është një bibliotekë për krijimin e grafikëve dhe diagrameve **dy-dimensionale** në Python. Nuk është pjesë e librarisë standarde të Python-it, prandaj duhet ta instalosh veçmas *pasi* ke instaluar Python në sistemin tënd.

Për ta instaluar në **Windows**, hap një dritare Command Prompt dhe shkruaj:

```
pip install matplotlib
```

Në **macOS** ose **Linux**, hap një Terminal dhe shkruaj:

```
sudo pip3 install matplotlib
```

1.2 Importimi i modulit pyplot

Paketa `matplotlib` përmban një modul të quajtur `pyplot`, të cilin do ta importosh për të krijuar të gjithë grafikët e demonstruar në këtë kapitull. Ka disa teknika importimi.

1.2.1 Forma e drejtpërdrejtë

```
import matplotlib.pyplot
```

Me këtë formë, çdo thirrje funksioni duhet të ketë parashtesën `matplotlib.pyplot`. P.sh. funksioni `plot` për grafikë vijorë thirret kështu:

```
matplotlib.pyplot.plot(arguments...)
```

1.2.2 Me alias (praktika më e përdorur)

```
import matplotlib.pyplot as plt
```

Kjo krijon aliasin `plt` për modulin `matplotlib.pyplot`, ndaj mund të thërrasësh funksionet me parashtesën e shkurtër `plt`:

```
plt.plot(arguments...)
```

1.3 Grafik vijor (Line Graph) me plot

Grafiku vijor është një nga format më të thjeshta dhe më të përdorura për vizualizimin e të dhënave numerike. Ai lidh një seri pikash të dhënash me vija të drejta, duke treguar ndryshimin e një variabli (Y) në varësi të një tjetri (X). Boshti horizontal (**X**) zakonisht përfaqëson kategorinë, kohën ose një variabël të pavarur, ndërsa boshti vertikal (**Y**) përfaqëson vlerat e matura.
Për të ndërtuar një grafik vijor, duhet të përgatiten dy lista: njëra për vlerat X dhe tjetra për vlerat Y. Funksioni kryesor për vizatimin e vijës është `plot()`, ndërsa `show()` përdoret për të shfaqur grafikën në ekran.

1.3.1 Grafiku 1: Shembull bazë

Shembull i pikave:

```
(0, 0), (1, 3), (2, 1), (3, 5), (4, 2)
```

Listat përkatëse:

```
x_coords = [0, 1, 2, 3, 4]
y_coords = [0, 3, 1, 5, 2]
```

Thirrjet bazë:

```
plt.plot(x_coords, y_coords)
plt.show()
```

Programi 1 (line_graph1.py)

```python
import matplotlib.pyplot as plt
def main():
    # Krijon listat me koordinatat X dhe Y të çdo pike të dhënash.
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    # Ndërton grafikën vijore.
    plt.plot(x_coords, y_coords)

    # Shfaq grafikën vijore.
    plt.show()

if __name__ == '__main__':
    main()
```

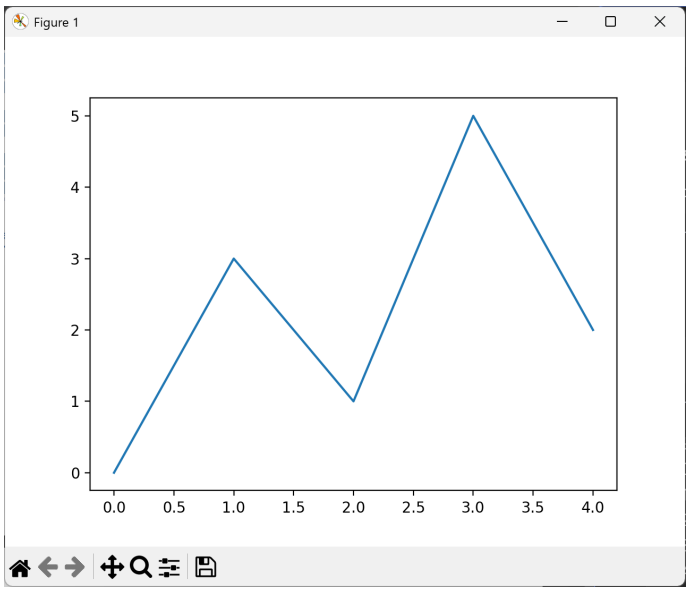

Figure 1.1: Shembull i një grafiku vijor të krijuar me matplotlib.

Funksionet e reja:

- `plot(x, y)` : Vizaton një vijë që lidh pikat me koordinata të dhëna nga listat `x` dhe `y`. Kjo është metoda bazë për grafik vijor.
- `show()` : Hap dritaren grafike dhe shfaq grafikën. Pa këtë thirrje, grafiku nuk do të shfaqet vizualisht.

1.3.2 Grafike 2: Shtimi i titullit, etiketave dhe rrjetës

Grafiku vijor mund të përmirësohet duke shtuar titull, etiketa për boshtet dhe rrjetë për lehtësi leximi. Këto elemente e bëjnë grafikën më të kuptueshme dhe më të lexueshme.

Programi 2 (line_graph2.py)

```python
import matplotlib.pyplot as plt
def main():
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    plt.plot(x_coords, y_coords)

    # Shto një titull.
    plt.title('Sample Data')

    # Shto etiketa për boshtet.
    plt.xlabel('This is the X axis')
    plt.ylabel('This is the Y axis')

    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Funksionet e reja:

- `title(str)` : Vendos një titull në krye të grafikës.
- `xlabel(str)` , `ylabel(str)` : Vendosin etiketa për boshtin horizontal (X) dhe vertikal (Y).
- `grid(True)` : Shton rrjetë (vija horizontale dhe vertikale të lehta) për të ndihmuar në leximin e vlerave.

1.3.3 Grafiku 3: Përshtatja e kufijve të boshtit X dhe Y

Boshtet X dhe Y kanë kufij të paracaktuar që varen nga vlerat minimale dhe maksimale të të dhënave. Për të kontrolluar manualisht intervalin e shfaqur, përdoren funksionet `xlim()` dhe `ylim()` .

```python
plt.xlim(xmin=1, xmax=100)
plt.ylim(ymin=10, ymax=50)
```

Programi 3 (line_graph3.py)

```python
import matplotlib.pyplot as plt
def main():
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    plt.plot(x_coords, y_coords)

    plt.title('Sample Data')
    plt.xlabel('This is the X axis')
    plt.ylabel('This is the Y axis')

    # Vendos kufijtë e boshtit.
    plt.xlim(xmin=-1, xmax=10)
    plt.ylim(ymin=-1, ymax=6)

```

```
    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Funksionet e reja:

- `xlim(xmin, xmax)` : Përcakton kufijtë e boshtit X.
- `ylim(ymin, ymax)` : Përcakton kufijtë e boshtit Y.

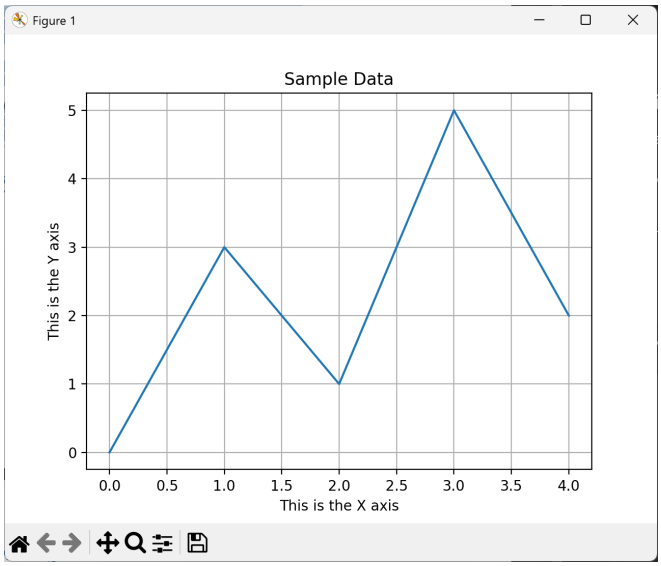

Figure 1.2: Grafiku vijor me titull, etiketa të boshtit dhe rrjetë.

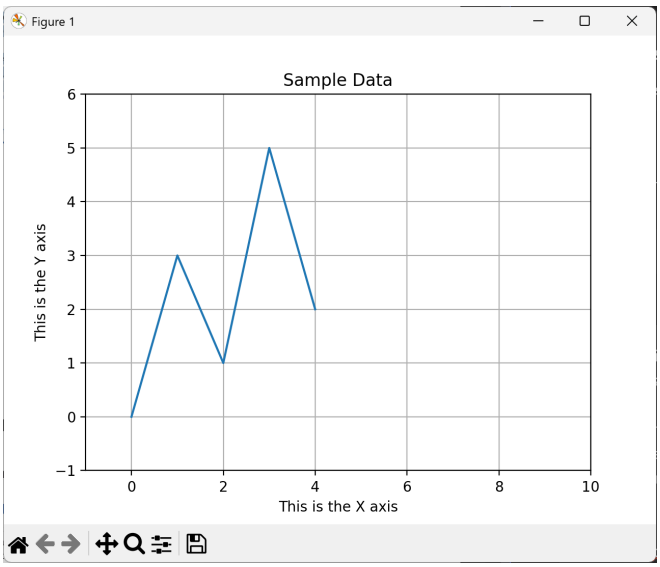

Figure 1.3: Grafiku vijor me kufij të personalizuar për boshtet X dhe Y.

1.3.4 Grafiku 4: Përshtatja e tick-ëve me `xticks` / `yticks`

Në grafikët vijorë, shenjat e ndarjes (ticks) në boshtet X dhe Y mund të personalizohen për të treguar kategori ose etiketa të veçanta. Kjo është veçanërisht e dobishme kur boshti X përfaqëson vite, kategori, ose emra.

```
plt.xticks([0, 1, 2], ['Baseball', 'Basketball', 'Football'])
plt.yticks([0, 1, 2, 3], ['Zero', 'Quarter', 'Half', 'Three Quarters'])
```

Programi 4 (`line_graph4.py`)

```
import matplotlib.pyplot as plt
def main():
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    plt.plot(x_coords, y_coords)
    plt.title('Sales by Year')

    plt.xlabel('Year')
    plt.ylabel('Sales')
```

```python

    # Personalizo shenjat e ndarjes.
    plt.xticks([0, 1, 2, 3, 4], ['2016', '2017', '2018', '2019', '2020'])
    plt.yticks([0, 1, 2, 3, 4, 5], ['$0m', '$1m', '$2m', '$3m', '$4m', '$5m'])

    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Funksionet e reja:

- `xticks(positions, labels)` : Vendos pozicionet dhe etiketat për shenjat e ndarjes në boshtin X.
- `yticks(positions, labels)` : Vendos pozicionet dhe etiketat për shenjat e ndarjes në boshtin Y.

1.3.5 Grafiku 5: Vendosja e markuesve (markers) në pikat e të dhënave

Për të theksuar çdo pikë të dhënash në grafik, përdoren markues (markers). Këto janë simbole të vogla që vendosen në secilën pikë të grafikut. Markuesit mund të ndryshojnë formën sipas nevojës.

Programi 5 (`line_graph5.py`)

```python
import matplotlib.pyplot as plt
def main():
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    plt.plot(x_coords, y_coords, marker='o')
    plt.title('Sales by Year')

    plt.xlabel('Year')
    plt.ylabel('Sales')

    # Personalizo shenjat e ndarjes.
    plt.xticks([0, 1, 2, 3, 4], ['2016', '2017', '2018', '2019', '2020'])
    plt.yticks([0, 1, 2, 3, 4, 5], ['$0m', '$1m', '$2m', '$3m', '$4m', '$5m'])

    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Funksionet e reja:

- `marker` : Argument opsional te `plot()` që përcakton formën e markuesit për çdo pikë të dhënash. Disa vlera të zakonshme janë: `'o'` : rreth, `'s'` : katror, `'^'` : trekëndësh lart, `'v'` : trekëndësh poshtë, `'*'` : yll, `'D'` : romb

Argumenti pozicional i markuesit: Nëse karakteri i markuesit jepet si argument pozicional (p.sh. `'o'`), atëherë vizatohen vetëm markuesit pa vija lidhëse. Kjo është e dobishme kur dëshiron të shfaqësh vetëm pikat, jo linjën.

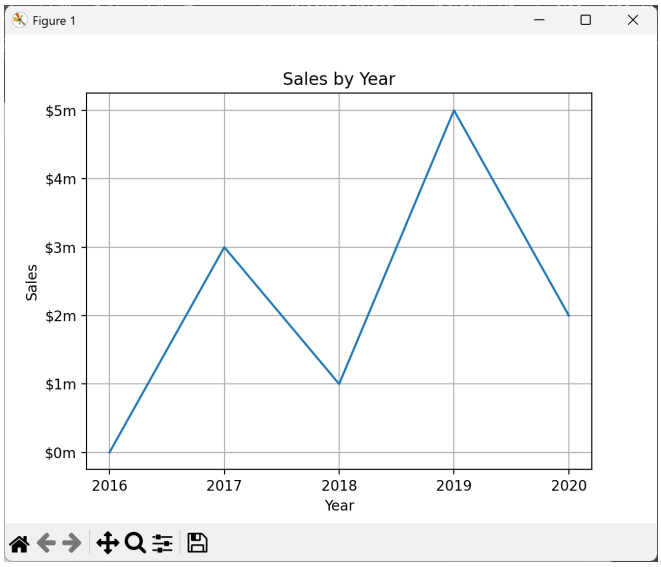

Figure 1.4: Grafiku vijor me etiketa të personalizuara për boshtet dhe tick marks.

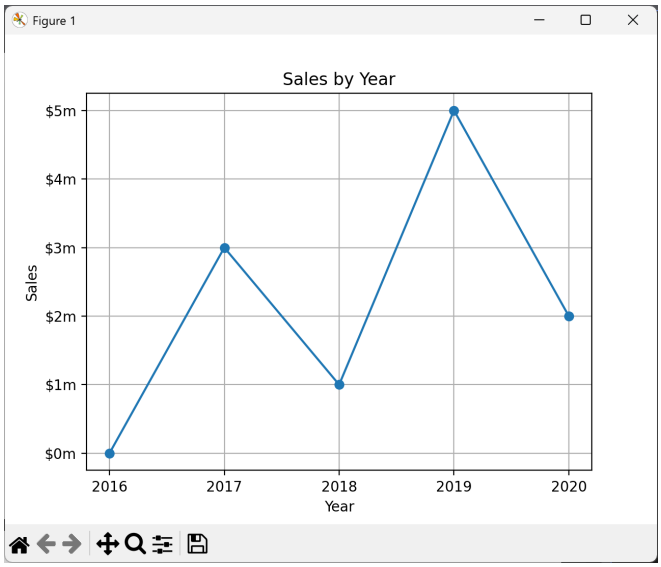

Figure 1.5: Grafiku vijor me markera të rrumbullakët në çdo pikë të dhënash.

```
plt.plot(x_coords, y_coords, 'o')
```

1.3.6 Grafiku 5 dhe 6: Shembuj shtesë me `annotate`, `color`, `linestyle`

Matplotlib lejon personalizime të mëtejshme si anotime (shënime) në pika të veçanta, ndryshim të ngjyrës së vijës dhe stilit të saj.

Programi 6 (`line_graph6.py`)

```
import matplotlib.pyplot as plt
def main():
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    plt.plot(x_coords, y_coords, marker='o')
    plt.title('Sales by Year')

    plt.xlabel('Year')
    plt.ylabel('Sales')

    plt.xticks([0, 1, 2, 3, 4], ['2016', '2017', '2018', '2019', '2020'])
    plt.yticks([0, 1, 2, 3, 4, 5],['$0m', '$1m', '$2m', '$3m', '$4m', '$5m'])

    # Anotim i një pike specifike në grafikun.
    plt.annotate('Highest Sales', xy=(3, 5), xytext=(2.5, 4.5), arrowprops={'arrowstyle': '->'
        })

    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Funksionet e reja:

- `annotate(text, xy, xytext, arrowprops)` : Shton një shënim (tekst) në një pikë të caktuar të grafikut. Argumenti `xy` tregon koordinatat e pikës që do të theksohet, ndërsa `xytext` është pozicioni i tekstit. `arrowprops` lejon vizatimin e një shigjete nga teksti te pika.

Programi 7 (`line_graph7.py`)

```python
import matplotlib.pyplot as plt
def main():
    x_coords = [0, 1, 2, 3, 4]
    y_coords = [0, 3, 1, 5, 2]

    plt.plot(x_coords, y_coords, marker='o', color='green', linestyle='--')
    plt.title('Sales by Year')

    plt.xlabel('Year')
    plt.ylabel('Sales')

    plt.xticks([0, 1, 2, 3, 4], ['2016', '2017', '2018', '2019', '2020'])
    plt.yticks([0, 1, 2, 3, 4, 5],['$0m', '$1m', '$2m', '$3m', '$4m', '$5m'])

    plt.annotate('Highest Sales', xy=(3, 5), xytext=(2.5, 4.5), arrowprops={'arrowstyle': '->'
        })
    plt.grid(True)
    plt.show()

if __name__ == '__main__':
    main()
```

Funksionet e reja:

- `color` : Argument opsional te `plot()` që përcakton ngjyrën e vijës (p.sh. `'green'`, `'red'`, `'blue'`).
- `linestyle` : Argument opsional te `plot()` që përcakton stilin e vijës (p.sh. `'-'` për vija të ndërprera, `'-'` për vija të plota).

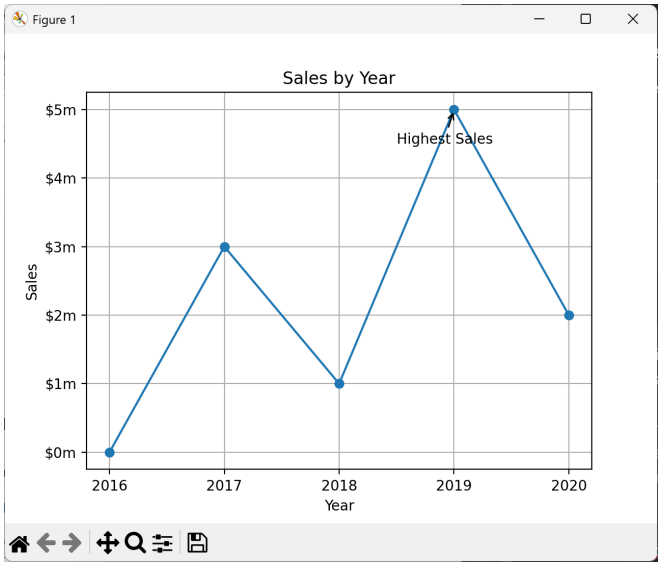

Figure 1.6: Grafiku vijor me markera, anotim, ngjyrë të personalizuar dhe stil të ndryshëm vije.

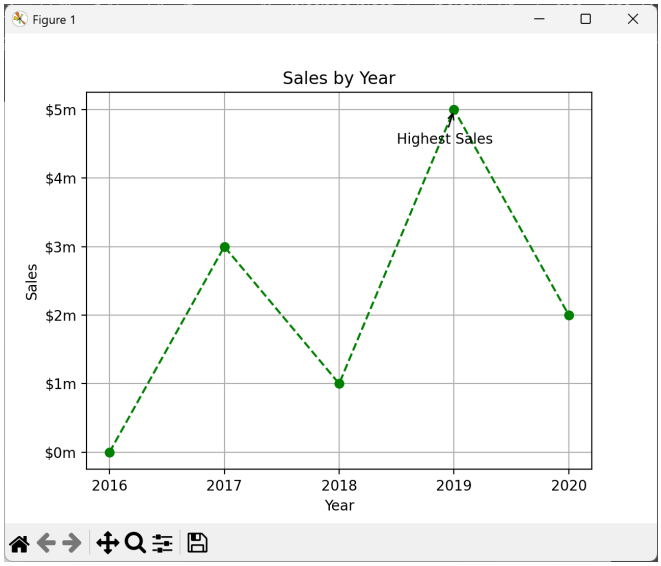

Figure 1.7: Grafiku vijor me markera, anotim, ngjyrë të personalizuar dhe stil të ndryshëm vije.

1.4 Grafik me kolona (Bar Chart)

Grafiku me kolona (**bar chart**) është një mënyrë shumë e përdorur për të krahasuar vlera të ndryshme, zakonisht midis kategorive. Në këtë grafik, çdo kolonë përfaqëson një kategori, dhe lartësia e kolonës tregon vlerën numerike të asaj kategorie. Boshti horizontal (**X**) përmban kategoritë, ndërsa boshti vertikal (**Y**) tregon vlerat. Bar chart është i dobishëm kur duam të shohim ndryshimet midis grupeve, të krahasojmë të dhëna të pavarura, ose të paraqesim të dhëna të mbledhura sipas kategorive (p.sh. shitjet sipas viteve, numri i studentëve sipas fakulteteve, etj.). Për të ndërtuar një bar chart me `matplotlib.pyplot.bar`, duhet të përgatiten dy lista:

- Një listë me pozicionet horizontale të kolonave (zakonisht skaji i majtë i çdo kolone).
- Një listë me lartësitë e kolonave, që përfaqësojnë vlerat për secilën kategori.

Programi 8 (bar_chart1.py)

```
import matplotlib.pyplot as plt

def main():
        left_edges = [0, 10, 20, 30, 40]
        heights = [100, 200, 300, 400, 500]

        plt.bar(left_edges, heights)
        plt.show()

if __name__ == '__main__':
        main()
```

Funksion i ri:

- `bar(x, height)`: Krijon një grafik me kolona. Lista `x` tregon pozicionin horizontal të secilës kolonë, ndërsa `height` tregon lartësinë e secilës kolonë.

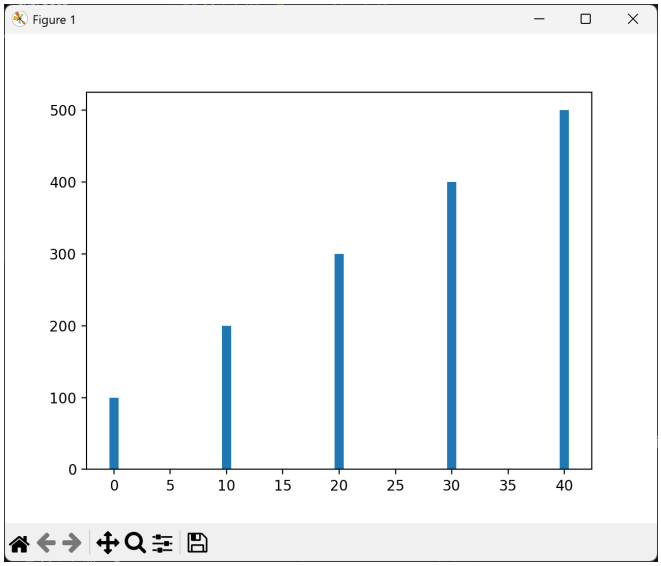

Figure 1.8: Grafik me kolona (bar chart) me gjerësi standarde.

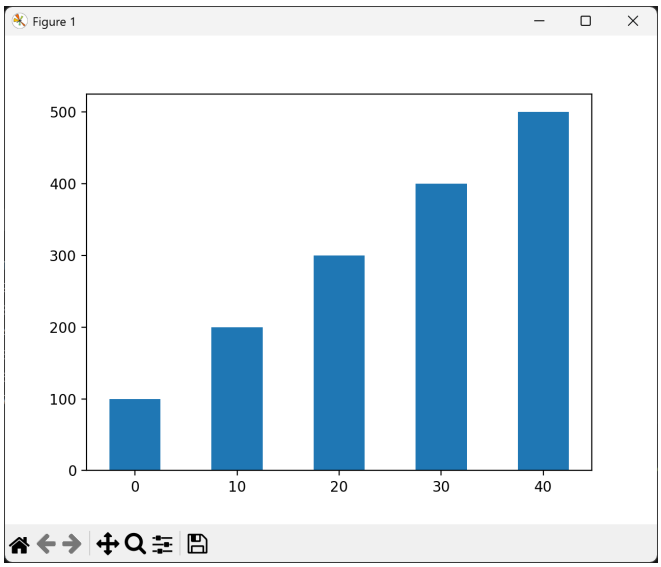

Figure 1.9: Grafik me kolona (bar chart) me gjerësi të personalizuar.

1.4.1 Gjerësia e kolonave (*bar width*)

Secila kolonë ka një gjerësi të caktuar përgjatë boshtit X. Parazgjedhja është `0.8`, por mund ta ndryshosh duke kaluar një argument të tretë te funksioni `bar`. Kjo është e dobishme kur dëshiron që kolonat të jenë më të gjera ose më të ngushta, ose kur ke nevojë të përshtatësh hapësirën midis tyre.

Programi 9 (bar_chart2.py)

```
import matplotlib.pyplot as plt

def main():
        left_edges = [0, 10, 20, 30, 40]
        heights = [100, 200, 300, 400, 500]

        # Gjerësia e kolonave (këtu më e madhe se parazgjedhja 0.8).
        bar_width = 5

        plt.bar(left_edges, heights, bar_width)
        plt.show()

if __name__ == '__main__':
        main()
```

Funksion i ri:

- Argumenti `width` te `bar()`: Përcakton gjerësinë e secilës kolonë përgjatë boshtit X.

1.4.2 Ndryshimi i ngjyrave të kolonave

Ngjyra e kolonave mund të personalizohet duke përdorur parametrin `color`. Mund të kalosh një tuple me kode ngjyrash, ku secila ngjyrë i korrespondon një kolone. Kjo e bën grafikën më të lexueshme dhe ndihmon në dallimin e kategorive.

```
plt.bar(left_edges, heights, color=('r', 'g', 'b', 'w', 'k'))
```

Funksion i ri:

- Argumenti `color` te `bar()`: Lejon të përcaktosh ngjyrën e secilës kolonë duke kaluar një tuple me kode ngjyrash.

1.4.3 Titulli, etiketat dhe *tick*-ët e personalizuar

Bar chart mund të përmirësohet duke shtuar titull, etiketa për boshtet dhe tick marks të personalizuar. Këto elemente e bëjnë grafikën më të kuptueshme dhe më të lehtë për t'u lexuar. Tick marks në boshtin X zakonisht vendosen në qendrat e kolonave, që llogariten si `left_edge + bar_width/2`.

Programi 10 (bar_chart3.py)

```
import matplotlib.pyplot as plt

def main():
        left_edges = [0, 10, 20, 30, 40]
        heights = [100, 200, 300, 400, 500]
```

```
6        bar_width = 10
7
8        # Ndërto grafikun dhe cakto ngjyrat për secilën kolonë.
9        plt.bar(left_edges, heights, bar_width, color=('r', 'g', 'b', 'w', 'k'))
10
11       # Shto titull dhe etiketa boshte.
12       plt.title('Sales by Year')
13       plt.xlabel('Year')
14       plt.ylabel('Sales')
15
16       # Tick-ët e personalizuar:
17       # Vlerat [5, 15, 25, 35, 45] janë qendrat e kolonave kur bar_width = 10.
18       plt.xticks([5, 15, 25, 35, 45], ['2016', '2017', '2018', '2019', '2020'])
19
20       # Etiketa për boshtin Y në hapa 100-sh.
21       plt.yticks([0, 100, 200, 300, 400, 500], ['$0m', '$1m', '$2m', '$3m', '$4m', '$5m'])
22
23       plt.show()
24
25   if __name__ == '__main__':
26       main()
```

Funksione të reja:

- `title(str)` : Vendos një titull në krye të grafikës.
- `xlabel(str)` , `ylabel(str)` : Vendosin etiketa për boshtet X dhe Y.
- `xticks(positions, labels)` : Vendos pozicionet dhe etiketat për tick marks në boshtin X.
- `yticks(positions, labels)` : Vendos pozicionet dhe etiketat për tick marks në boshtin Y.

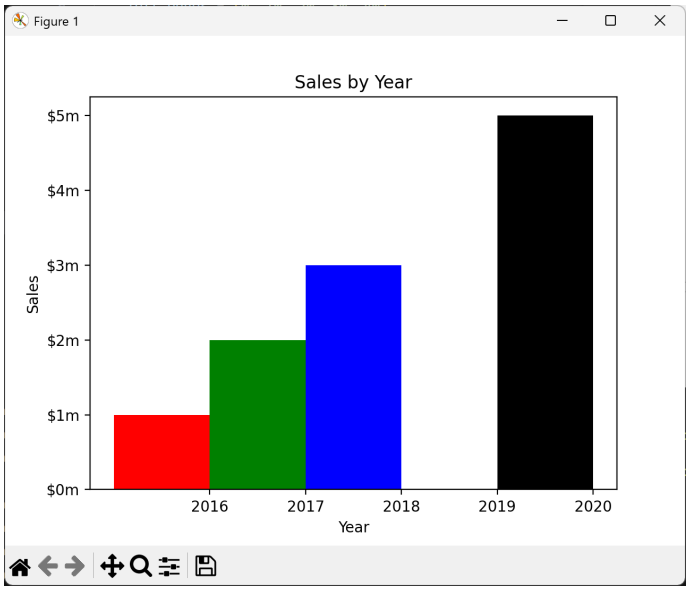

Figure 1.10: Shembull i një grafik bar chart me titull, etiketa, tick marks të personalizuar dhe ngjyra të ndryshme për çdo kolonë.

1.5 Grafik rrethor (Pie Chart)

Një **pie chart** është një grafik që paraqet të dhënat si pjesë të një tërësie, ku secili sektor i rrethit tregon përqindjen që zë një kategori ndaj totalit. Ky lloj grafiku është i përshtatshëm kur duam të shohim ndarjen e një sasie në pjesë përbërëse, si p.sh. ndarja e shitjeve sipas tremujorëve, shpenzimet sipas kategorive, etj.

Për të ndërtuar një pie chart me `matplotlib`, përdoret funksioni `pie()` nga moduli `matplotlib.pyplot`. Ky funksion merr si argument një listë vlerash numerike, të cilat përfaqësojnë madhësinë e secilit sektor. Shuma e këtyre vlerave përfaqëson tërësinë (100%).

Programi 11 (`pie_chart1.py`)

```python
import matplotlib.pyplot as plt

def main():
    values = [20, 60, 80, 40]

    # Ndërto grafikun rrethor nga vlerat.
    plt.pie(values)

    plt.show()

if __name__ == '__main__':
    main()
```

Funksion i ri:

- `pie(values)` : Krijon një grafik rrethor ku secili element i listës `values` përfaqëson madhësinë e një sektori. Sektori është proporcional me vlerën e tij ndaj shumës totale.

Në këtë shembull, rrethi ndahet në katër sektorë, secili sipas vlerës së dhënë. Përqindja e secilit sektor llogaritet automatikisht nga matplotlib.

1.5.1 Etiketa të sektorëve dhe titulli i grafikut

Për ta bërë grafikën më të kuptueshme, mund të shtojmë etiketa për secilin sektor duke përdorur parametrin `labels`. Kjo është e dobishme kur duam të tregojmë emrin e kategorisë që përfaqëson çdo pjesë e rrethit. Titulli i grafikut vendoset me funksionin `title()`.

Programi 12 (`pie_chart2.py`)

```python
import matplotlib.pyplot as plt

def main():
    sales = [100, 400, 300, 600]
    slice_labels = ['1st Qtr', '2nd Qtr', '3rd Qtr', '4th Qtr']

    # Ndërto grafikun rrethor me etiketa.
    plt.pie(sales, labels=slice_labels)

    plt.title('Sales by Quarter')
    plt.show()

if __name__ == '__main__':
    main()
```

Funksione të reja:

- `labels` : Parametër opsional te `pie()` , që lejon të shfaqen etiketa për secilin sektor të grafikut.
- `title(str)` : Vendos një titull në krye të figurës.

1.5.2 Ndryshimi i ngjyrave të sektorëve

Matplotlib cakton ngjyrat e sektorëve automatikisht, por mund të përcaktojmë ngjyrat manualisht me parametrin `colors` . Kjo është e dobishme kur duam të dallojmë më qartë kategoritë ose të përdorim një paletë të caktuar ngjyrash.

```
plt.pie(values, colors=('r', 'g', 'b', 'w', 'k'))
```

Funksion i ri:

- `colors` : Parametër opsional te `pie()` , që lejon të përcaktohen ngjyrat e secilit sektor duke kaluar një tuple me kode ngjyrash.

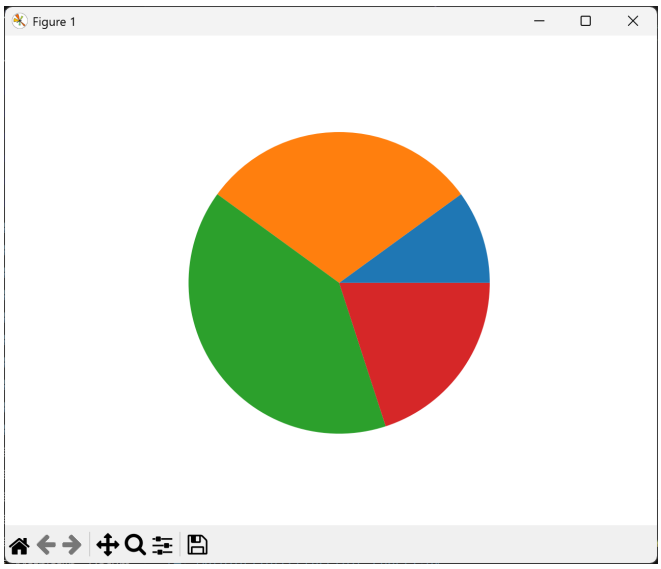

Figure 1.11: Grafik rrethor pa etiketa, vetëm me vlera.

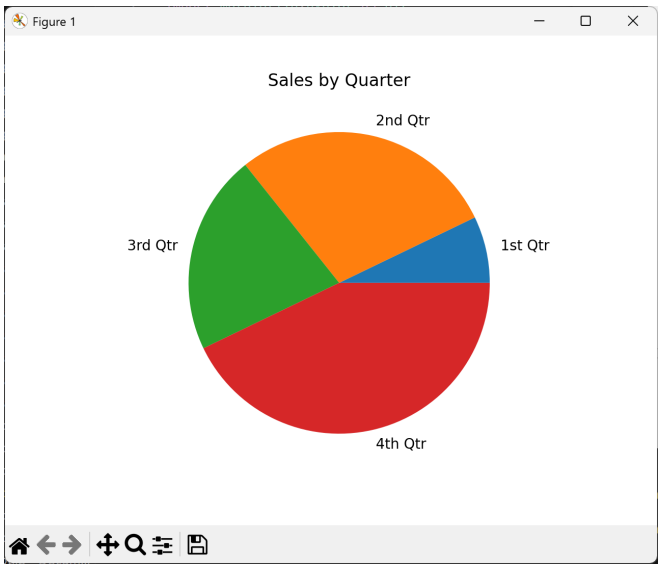

Figure 1.12: Grafik rrethor me etiketa për çdo sektor dhe titull.

1.6 Ndarja e figurës në nënfigura (Subplots)

Shpesh është e nevojshme të paraqesim disa grafikë të ndryshëm në të njëjtën figurë, për të krahasuar të dhënat ose për të vizualizuar rezultate të ndryshme krah për krah. Matplotlib mundëson ndarjen e një figure në disa nënfigura, të quajtura **subplots**. Çdo subplot është një zonë e veçantë brenda figurës, ku mund të vizatohet një grafik i pavarur.

Funksioni kryesor për këtë qëllim është `subplot(nrows, ncols, index)` , ku:

- `nrows`: numri i rreshtave të nënfigurave.
- `ncols`: numri i kolonave të nënfigurave.
- `index`: pozicioni i nënfigurës (numërohet nga 1).

Shembulli më poshtë tregon si të vendosim dy grafikë në një rresht:

Programi 13 (subplots1.py)

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [i**2 for i in x]
y3 = [i**3 for i in x]

plt.subplot(1, 2, 1) # 1 rresht, 2 kolona, grafiku i parë
plt.plot(x, y, 'r--', linewidth=5.0)
plt.title("Funksioni 1")

plt.subplot(1, 2, 2) # 1 rresht, 2 kolona, grafiku i dytë
plt.plot(x, y3, 'g^-', linewidth=1.0)
plt.title("Funksioni 2")

plt.savefig("filename2.png") # Ruaj figurën si PNG
plt.show()
```

Funksione të reja:

- `subplot(nrows, ncols, index)` : Ndarja e figurës në nënfigura sipas një grilë të caktuar.
- `savefig(filename)` : Ruajtja e figurës në një skedar (p.sh. PNG, PDF).

Në këtë shembull, figura ndahet në dy pjesë horizontale. Secila pjesë ka titullin dhe grafikën e vet.

Nëse duam të paraqesim më shumë nënfigura, mund të përdorim një grilë më të madhe. Shembulli më poshtë tregon si të vendosim katër grafikë në një figurë (2 rreshta, 2 kolona):

Programi 14 (subplots2.py)

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [i**2 for i in x]
y3 = [i**3 for i in x]

plt.subplot(2, 2, 1) # 2 rreshta, 2 kolona, grafiku i parë
plt.plot(x, y, 'r--', linewidth=5.0)
plt.title("Funksioni 1")

plt.subplot(2, 2, 2) # 2 rreshta, 2 kolona, grafiku i dytë
plt.plot(x, y3, 'g^-', linewidth=1.0)
plt.title("Funksioni 2")

plt.subplot(2, 2, 3) # 2 rreshta, 2 kolona, grafiku i tretë
plt.plot(x, [i ** 3 for i in x], 'b*-', linewidth=2.0)
plt.title("Funksioni 3")

plt.subplot(2, 2, 4) # 2 rreshta, 2 kolona, grafiku i katërt
plt.plot(x, [i ** 0.5 for i in x], 'ms-', linewidth=3.0)
plt.title("Funksioni 4")

plt.show()
```

Në këtë rast, figura ndahet në katër zona të barabarta. Secila nënfigurë ka grafikën dhe titullin e vet, duke lejuar krahasimin e rezultateve të ndryshme në të njëjtën pamje.

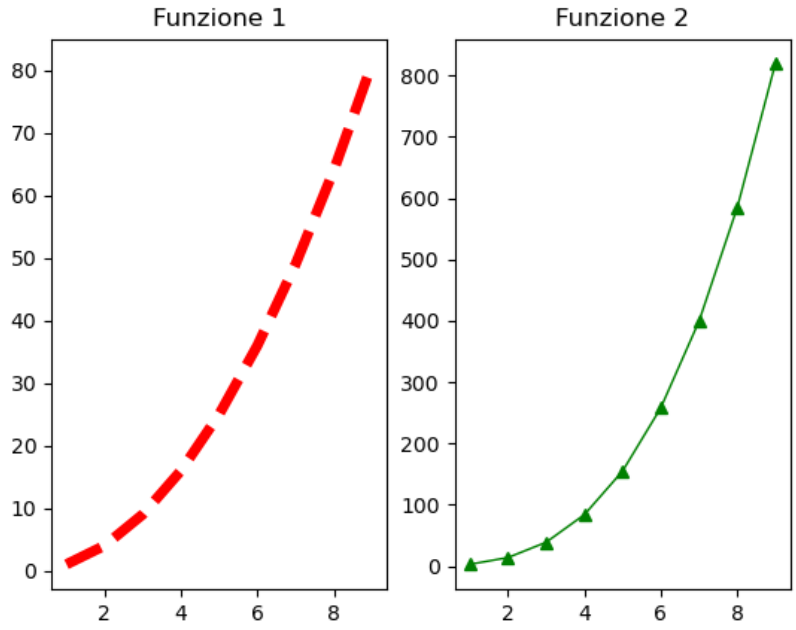

Figure 1.13: Dy nënfigura të vendosura në një rresht me `subplot`.

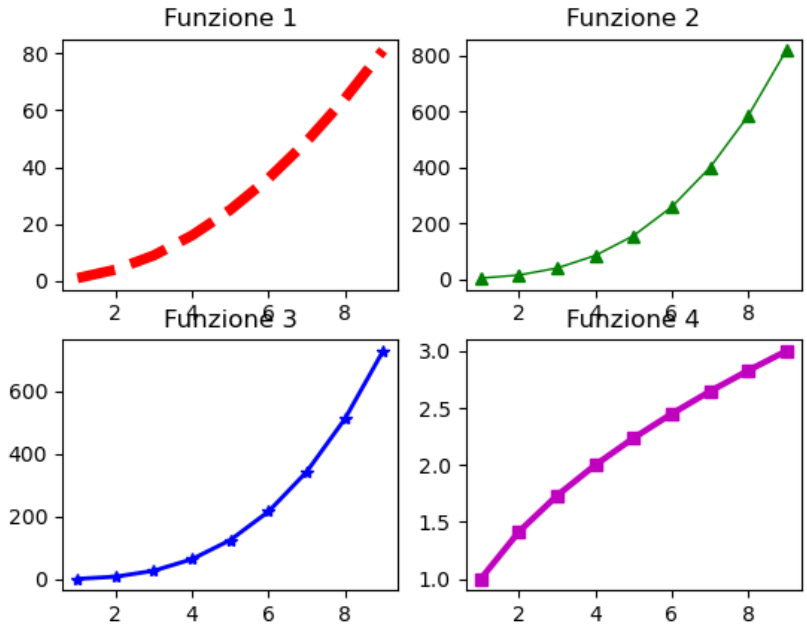

Figure 1.14: Katër nënfigura të vendosura në një figurë me `subplot`.

Çfarë mësuam në këtë kapitull

- **Krijimi i grafikëve**: `plot()`, `show()` dhe shtimi i titullit/etiketave/grilës me `title`, `xlabel`, `ylabel`, `grid`.
- **Kontrolli i pamjes**: `xlim/ylim` për kufijtë dhe `xticks/yticks` për tick-ët e personalizuar.
- **Stilizimi**: markera (`marker='o'` etj.), ngjyra (`color`), stil vije (`linestyle`) dhe trashësi (`linewidth`).
- **Bar Chart**: ndërtimi me `bar()`, përdorimi i `width` dhe `color`, si dhe etiketime të boshtit/tick-ëve.
- **Pie Chart**: ndërtimi me `pie()` dhe përdorimi i `labels`, `colors` dhe `title()`.
- **Subplots**: organizimi i shumë grafikëve në një figurë me `subplot()` dhe ruajtja e figurave me `savefig()`.